

TOWARD AN ANALYST-ORIENTED POLYSTORE FRAMEWORK FOR PROCESSING JSON GEO-DATA

Giuseppe Psaila and Paolo Fosci

University of Bergamo, Viale Marconi 5 – 24044 Dalmine (BG) - Italy

ABSTRACT

Geo-data sets represented as JSON documents are provided by many sources over the Internet. Consequently, many storage technologies are adopted for managing them, namely NoSQL databases such as MongoDB and ElasticSearch.

To allow analysts to effectively perform complex analysis processes, such as integration and transformations possibly based on spatial properties, a framework that provides a unified view of different database systems based on different technology is necessary, so that analysts are unaware of specific characteristics of different database systems. In other words, we follow a polystore approach.

The presented framework, named J-CO, provides a query language and an execution engine that, together, enable analysts to specify complex transformation processes on JSON geo-data sets. High-level operator natively support spatial operations, and completely hide technical details of the integrated database systems. This way, a novel data store component, named J-CO-DS, can be seamlessly introduced, to overcome limitations characterizing other NoSQL database systems. The paper presents the framework and the novel features (w.r.t. our previous works); then, it shows the effectiveness of the framework itself through an example.

KEYWORDS

JSON Geo-Data Sets, Nosql Database Systems, Analyst-Oriented Framework, Polystore Framework, High-Level and Spatial Enabled Query Language

1. INTRODUCTION

In the era of Big Data and Open Data, the representation format that is imposing as the de-facto standard is JSON (JavaScript Object Notation, officially defined in Bryan and Nottingham (2013)). Due to its flexibility and simplicity (Smith (2015)), this format to serialize objects is becoming very popular. It is used in a large variety of application contexts, such as APIs of web services, both of information systems and of social media (such as Facebook), but it is also used to distribute Open Data sets through Open Data portals. The idea behind Open Data is that public administrations and analysts publish data sets concerning any aspect of life and citizenship in the territories: the free availability of such data sets can drive the creation of both economic and social value (Latif et al. (2009)). One of the most favorite format for open data sets is JSON.

Since JSON data sets often describe data concerning territories, or describe places visited or to be visited by people, frequently they are geo-referenced, i.e., they associate data with positions or areas on the Earth Globe. In this case, we call them JSON geo-data (sets).

As an effect, the GIS (Geographical Information System) community has been involved in this phenomenon as well and developed an open standard named GeoJSON to represent information layers by exploiting the JSON syntactic framework (Butler et al. (2016)). The consequences are twofold: GIS software now supports GeoJSON to import/export layers; many Open Data portals now publish geo-referenced data sets as GeoJSON files.

Obviously, the research area of database management has considered the advent of JSON as well. In particular, novel systems belonging to the category of NoSQL databases, i.e., databases that do not rely on the relational data model and, consequently,, are not based on SQL as query language, have been proposed, which manage collections of JSON objects in a native way (Gessert et al. (2017), Han et al. (2011), Nayak et al. (2013)). In particular, we mention MongoDB (Banker2011) and ElasticSearch (Gormley and Tong

(2015)): the former can be considered a pure DBMS, the latter is a cross between a DBMS and an information retrieval system.

Based on the previous considerations, we can expect that analysts working on many geo-data sets coming from various data sources and represented as JSON documents use, at the same time, databases managed by MongoDB, databases managed by ElasticSearch (for performing complex text-based searches on data sets) and file systems. Such a mess of data storage systems can be a strong obstacle for analysts to access data sets and integrate them in a flexible and seamless way: a unifying framework is necessary.

This paper presents a novel analyst-oriented framework, named J-CO (*JSON CO*llections) to realize a kind of polystore database system (a concept introduced in Duggan et al. (2015)) for JSON geo-data sets. The idea is that the framework provides access to MongoDB databases, ElasticSearch databases and file systems. In particular, we developed a novel tool, named J-CO-DS, for managing JSON files stored on classical file systems as a database, in order to provide analysts with a unified view of storage systems.

The J-CO framework provides a powerful query language named J-CO-QL. This language, introduced in our previous works (Bordogna et al. (2017), Bordogna et al. (2017b), Bordogna et al. (2018)) is able to perform complex transformations of collections of JSON data, natively dealing with geo-tagging and spatial operations. In this paper, we will briefly present some features of the language, in order to show how the polystore approach enables analysts to easily perform complex transformations on geo-data sets, in such a way source and resulting geo-data sets are stored in databases powered by different technologies.

The paper is organized as follows. Section 2 presents the application scenario of the J-CO framework, by discussing the problem of getting geo-data sets from different database technology and by presenting the J-CO-DS novel component of the framework developed in this research. Section 3 presents an application example, where the J-CO-QL language is used to geo-code and transform social-media data with an institutional geographical layer and results are stored into an ElasticSearch database for further information retrieval activities. Finally, Section 4 draws the conclusions. For the sake of space limitation, references to background will be made through the paper.

2. J-CO POLYSTORE FRAMEWORK

In this section, we introduce the polystore view behind the J-CO framework. We start discussing the application context, then we show a scenario and we discuss the capabilities of the framework.

2.1 Application Context and Technical Scenario

Geo-data can be obtained from various sources with various semantics, depending on their content. For example, social media, such as Twitter, can be easily queried to get geo-referenced messages posted by users. Such messages typically say that the posting users were in the places denoted by geo-tagging (typically, a point on the Earth Globe) and describe impressions usually concerning those places. Consequently, they can provide analysts with an impressive amount of information, provided that these messages are properly processed and analyzed. This is what the *FollowMe* project (see Cuzzocrea et al. (2015), Cuzzocrea et al. (2016)) is doing, i.e., gathering geo-located tweets posted by moving people since May 2015.

For example, it could be interesting to partition messages by the country they are posted from. In order to do that, an authoritative data set is required, i.e., a description of boundaries of countries. In practice, we can figure out that analyst's goal is to geo-code messages in order to associate them with the name of the country where they are posted from.

Such an apparently simple and obvious transformation can be very useful in order to better characterize messages: the resulting corpus of messages labeled with country name could be published in a portal that provides Information Retrieval capabilities (Chowdhury (2010)). In this way, the portal could provide potential users with the possibility of querying the corpus by performing full-text searches in which the country plays an explicit role. An example of search could be: "extract messages posted in Spain in which *Sagrada Familia* is mentioned".

Figure 1.a depicts the above-mentioned application scenario: both data from social media (a corpus of geo-located messages) and authoritative data describing boundaries of countries are processed by means of some processing tools. The result is the same corpus of messages, where each message is now geo-coded with the name of the country where the message was posted from. The geo-coded corpus is transferred to an IR portal that allows users to query the corpus as previously mentioned.

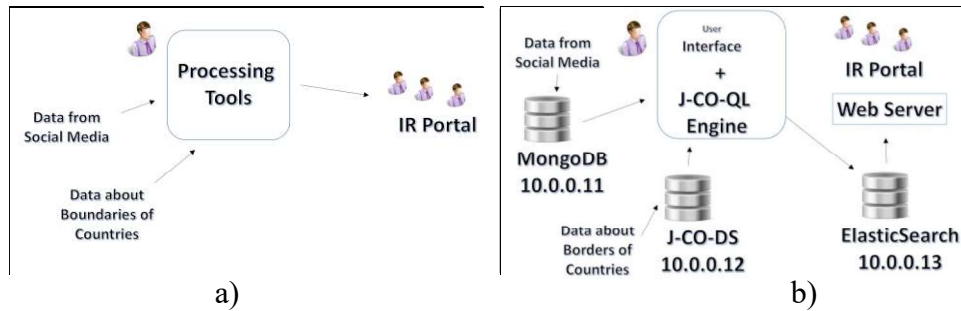


Figure 1. Perspective Application Scenario (a) and Technological Scenario (b)

Let us consider now the technological scenario behind the application scenario previously described. Two issues must be considered: which data formats are used to represent data sets? What technology to choose to manage such data sets?

Typically, social-media APIs provide data as JSON objects. Consequently, a corpus of messages could be seen as a large vector of JSON objects. In such a context, each object is relatively small (it corresponds to a message), but the data set could contain millions of objects. Consequently, the best choice to manage such a data set is to use a NoSQL database system like *MongoDB* (Banker (2011)). It has become very popular, because it natively deals with collections of JSON objects; no constraints about schema of data are posed, because the system is able to store objects with heterogeneous structure within the same collection. MongoDB has a major limitation: it does not accept objects whose internal representation (in BSON, the binary representation for JSON objects) exceeds 16Mbyte. This limitation is not a problem for messages from social media, that are much smaller.

As far as authoritative sources of geo-referenced data are concerned, there are many of them on Open Data portals: on them, many information layers (i.e., geo-referenced data sets) concerning territories are published by states and by local administrations. Usually, formats adopted for representing them are *Shape File* format (ESRI 1998) and, more recently, *GeoJSON* (Butler et al. (2016)). The advantage of using GeoJSON is that it is an open format that object-oriented programming languages can easily process. However, in the case of an information layer describing boundaries of countries all over the world, the resulting GeoJSON document can be very large; in particular, a GeoJSON information layer is a single-object document, in which a field named `features` contains as many objects as the geo-referenced features to describe. Consequently, if we have to decide how to store and manage it, immediately it appears that MongoDB is not suitable, because the BSON representation of a GeoJSON layer can exceed the limitation of 16 Mbyte easily.

For this reason, once downloaded, a simple solution is to store it as a file in the file system of a server.

If we consider the problem of building an IR portal on a data set, some tools provide an excellent support to web servers. One of the mostly-used tool is *ElasticSearch* (Gormley and Tong (2015)). It provides basic DBMS functionality for storing and managing data sets, while it provides excellent information retrieval functionality (like full-text search) to retrieve the so-called “mostly relevant documents”. This is done by means of sophisticated indexing techniques.

The data format to provide data sets to and to extract data sets from ElasticSearch is JSON; so, it can be seen again as a NoSQL database system for JSON documents.

The outcomes of the previous considerations are the issues described hereafter.

- Different NoSQL database systems such as MongoDB and ElasticSearch provide different query language with different capabilities.

- The support to perform spatial operations on stored data sets is different, possibly limited (in terms of capabilities) or even possibly absent.
- The integration of data sets requires to move data sets from one system to another; this is a significant waste of time for analysts.
- Large GeoJSON layers are not supported by MongoDB. Consequently, it appears that the only solution is to write computer programs in a procedural programming language. However, often analysts are not computer programmers or do not have time to spend in programming.
- The overall analysis and transformation process should be performed in a seamless way w.r.t. the specific NoSQL systems to work with and with which to exchange data sets.

Both the application scenario and the technical scenario we described can be considered the synthesis of what the authors had to face during the participation to a research project jointly conducted with geographers (Burini et al. (2018), Burini et al. (2018b)). The idea is to adopt a data-driven approach to study mobility of city users. The idea is to gather and integrate various data coming from social media and authoritative sources concerning public transportation systems and movements of social-media users. This project clarified us the current limitations of the application scenario and gave us the idea of developing the J-CO framework.

2.2 The J-CO Framework

The solution to the above-mentioned issues is a polystore framework that gives an analyst-oriented, high-level and unified view of several NoSQL database systems. So, we started the development of the J-CO framework (Bordogna et al. (2017), Bordogna et al. (2017b)). It is an ongoing project whose goal is to provide analysts (such as geographers) with a powerful tool that allows them to perform complex transformations of geo-data sets (represented as JSON documents) by means of high-level languages and tools. Currently, the J-CO framework has the following features.

- It provides a query language, named J-CO-QL, that natively deals with collections of heterogeneous JSON objects, providing high-level operators for spatial transformations. The language is independent of query languages provided by any NoSQL database system. The language was inspired by our previous works on database models for complex spatial information layers (Bordogna et al. (2006), Psaila (2011)).
- The J-CO-QL-Engine is the software component that processes J-CO-QL queries. It can connect to various NoSQL database systems at the same time, in a totally seamless way for users. J-CO-QL Engine is, therefore, a kind of polystore database system for JSON data sets. It is implemented by using the Java programming language.
- In order to provide a structured way to manage large JSON data sets (in particular, large GeoJSON information layers) stored in file systems, we developed the J-CO-DS component. It is a *data store* tool to organize previously existing JSON data sets and newly-generated data sets in a file system; J-CO-DS provides users and J-CO with the same abstractions (i.e., databases and collections) provided by MongoDB and ElasticSearch. Similarly to J-CO-QL Engine, J-CO-DS is implemented by using the Java programming language.
- A user interface, that is under development, will provide users with an interactive tool to compose complex J-CO-QL queries, by inspecting intermediate results and by possibly rolling back (or backtracking, but not in the sense of artificial intelligence) the query execution.

With the introduction of the J-CO framework, the application scenario previously discussed and shown in Figure 1.a becomes the technological scenario depicted in Figure 1.b.

- The data set obtained by querying social media (like Twitter) is stored into a database managed by MongoDB; the server is identified by the IP address 10.0.0.11.
- Authoritative data sets concerning boundaries of countries are stored into the file system of server with IP address 10.0.0.12. An instance of the J-CO-DS service is running on this server, providing a database view of stored data sets.
- An instance of ElasticSearch is running on server with IP address 10.0.0.13. Its goal is to store the corpus to publish on the IR portal; it also powers the portal with full-text search functionality. Either client-side scripts or server-side scripts (depending on the choice made by the designer of the portal) published by the Web Server submit queries to the ElasticSearch service.

- The J-CO-QL Engine is used by the analyst to integrate and export the data sets. In particular: social-media messages stored in the MongoDB database are retrieved and geo-coded with the GeoJSON layer describing boundaries of countries, stored on server 10.0.0.12; the geo-coded corpus of messages is stored into the ElasticSearch database, to be indexed and published on the IR portal.

The advantage of using the J-CO framework is that one single tool (the J-CO-QL Engine) and one single language (J-CO-QL) is used. No knowledge about specific query languages and API of each single NoSQL database system is needed. J-CO-QL provides operations for spatial transformations on the data: it is a high-level and analyst-oriented language, so that computer-programming skills are not necessary.

2.3 J-CO-DS, the File System-based Data Store

J-CO-DS is a data-store system based on the file system. Apparently, this definition could be contradictory, since the J-CO framework has been designed to work with NoSQL database systems. However, in the previous subsection we motivated the limitations to overcome, mainly the inability of MongoDB to store JSON objects with BSON representation larger than 16 MByte and the substantial inability of ElasticSearch to manipulate data sets in a complex way. Nevertheless, another need could be to build a unified database view of all data sets previously found in the Internet and stored into the file system.

Thus, the goal of J-CO-DS is to provide a database view of the file system. In fact, it provides the following functionality.

- J-CO-DS permits to create new databases. Collections in databases are sets of JSON objects.
- Collections are managed through the file system: the content of the same collection could be stored in different files.
- J-CO-DS is able to import previously-existing JSON files, incorporating them into collections.
- J-CO-DS is able to import a JSON file available as Internet resource.
- J-CO-DS is able to export collections into files external to J-CO-DS.

In short, the basic operations that are supported by J-CO-DS are the following ones.

- J-CO-DS permits to create (and delete) databases.
- J-CO-DS permits to create and delete collections within a database. Collections can be populated either by providing the content (as done by the J-CO-QL Engine) or by collating files from the file systems (by checking the compliance of them with the JSON format). Collections can also be deleted from databases.
- Collections can be retrieved and sent to the J-CO-QL Engine or exported.

This way, when analysts write and execute J-CO-QL queries, they are provided with a uniform view of three different data storage systems.

3. AN EXAMPLE: GEO-CODING TWITTER MESSAGES

We now illustrate the power of the J-CO framework as a polystore framework for performing high-level spatial transformations on JSON collections. Hereafter we report the J-CO-QL query that performs the transformation process previously described, i.e., geo-coding messages by social-media users with names of countries where they were posted from. We had this need when analyzing more than 2 million tweets gathered by the *FollowMe* suite (Cuzzocrea et al (2015), Cuzzocrea et al. (2016)).

Before we start describe the query, we shortly summarize the execution model of the query. A full description can be found in Bordogna et al. (2018).

Operators work on one or two *collections*, i.e., sets of JSON objects. They get collections from databases, generate new collections, save collections into databases. An operator starts from a given query-process state s_i and generates a new query-process state s_{i+1} . Essentially, a query-process state contains a *temporary collection* tc , i.e., $s=(tc)$. The idea is that a temporary collection possibly will be the input of the next operator. Thus, an operator receives a temporary collection as input and generates a new temporary collection as output.

```

USE DB AuthoritativeData
  ON SERVER J-CO-DS "http://10.0.0.12:6001";
USE DB SocialMedia
  ON SERVER MongoDB "http://10.0.0.11:2707";
USE DB Portal
  ON SERVER ELASTICSEARCH "http://10.0.0.13:9200";

GET COLLECTION CountryBoundaries@AuthoritativeData;

EXPAND
  UNPACK WITH .features AND .name="EU_Boundaries"
    ARRAY .features TO .feature
  DROP OTHERS;

FIND
  CASE WHERE WITH .feature.item.properties.name,
                  .feature.item.geometry
  GENERATE { .CountryName: .feature.item.properties.name }
    SETTING GEOMETRY .feature.item.geometry
  DROP OTHERS;

SPATIAL JOIN OF COLLECTIONS Messages@SocialMedia AS M, TEMPORARY AS C
  ON INCLUDED(LEFT)
  SET GEOMETRY LEFT
  CASE WHERE WITH .M, .C
  GENERATE { .postID: .M.postID, .userID: .M.userID,
            .date: .M.date, .text: .M.text,
            .country: .C.CountryName,
            .coordinates: .~geometry.coordinates }
    DROPPING GEOMETRY
  DROP OTHERS;

SAVE AS GeoCodedMessages@Portal;

```

As a preliminary explanation, notice the structure of field references: since we have to deal with nesting within JSON objects, a field reference is a path expression having the syntax $(\textit{fieldname})^+$. For example, the field specification

.feature.item.properties.name

means that, starting from the root object, we refer to field `feature`, that in turn contains field `item`, that in turn contains field `properties` that, finally, contains field `name`.

We are now ready to explain the query.

- The three `USE DB` operations specify the databases which to connect to. They do not change the temporary collection, that is empty at the beginning of the process.
- The `GET COLLECTION` operation gets the specified collection `CountryBoundaries` from database `AuthoritativeData`, exposed by the `J-CO-DS` service on server `10.0.0.12`. The collection becomes the new temporary collection. See the first item in Figure 2 for an excerpt of this collection. Note that the shown object is a GeoJSON layer.
- Operator `EXPAND` un-nests objects within arrays. In particular, the `UNPACK` clause selects the objects to expand, in this case those with field `features` and with name `EU_Boundaries`. Notice that we want to get one single object, i.e., the one describing the GeoJSON layer for EU Countries (shown as first item in Figure 2). For each object within the `features` array, a new external object is generated into the temporary collection; such objects take all fields of the original object and a new field, named `feature` (as specified by the `TO` clause). This field is, in turn, an object containing two fields: `item`, which contains the object originally contained within the array, and `position`, denoting the position of the object in the unpacked array. The second item in Figure 2 is an excerpt of the new temporary collection coming out from the `EXPAND` operator. At the end of the operator, notice the specification `DROP OTHERS`: it means that objects not selected by the `UNPACK` clause must be dropped.

- The `FIND` operator is used to select objects within the temporary collection and restructure the selected objects. In this case, the operator takes the temporary collection containing objects describing boundaries of EU countries, in order to generate a simplified version with one single field `CountryName`, i.e., the name of the country. The specification `SETTING GEOMETRY` says that the standard field `~geometry` (in J-CO-QL, the standard field for representing geo-tagging of objects, see Bordogna et al. (2018)) assumes the value of field `geometry` coming from the former GeoJSON layer (see the third item in Figure 2). In fact, J-CO-QL works on the same representation for geometries provided by GeoJSON.
Again, the `DROP OTHERS` specification means that all objects not selected by the `CASE WHERE` clause must be dropped.
- The set of simplified objects describing countries is used to geo-code messages. The `SPATIAL JOIN` operator takes two collections as input. The former one is named `Messages` and is taken directly from database `SocialMedia` (managed by MongoDB on server 10.0.0.11); the second one is the temporary collection describing countries and boundaries, so far obtained. These collections are aliased, respectively, as `M` and `C`. The fourth item in Figure 2 is an excerpt of collection `Messages`, aliased as `M`.
The `ON` clause actually performs the spatial join. Each object in collection `M` is paired with each object in collection `C`: the pair is valid if the geometry of the `LEFT` object (a message) is included in the geometry of the right object (a country). For each valid pair, an object with three fields is generated: field `M`, containing the original object coming from collection `M`, field `C`, containing the original object coming from collection `C`, the `~geometry` field, that contains the geometry of the left object (as specified by `SET GEOMETRY LEFT`). The fifth item in Figure 2 shows an excerpt of objects resulting from this first step in the `SPATIAL JOIN` operator.
At this point, the `CASE WHERE` clause behaves as in the `FIND` operator: the structure of objects is simplified, in order to have all fields coming from the message and the country name at the top-most level; furthermore, in order to be compliant with the IR portal, longitude and latitude of the message are extracted from within the `~geometry` field, as an array named `coordinates`. The last item in Figure 2 shows an excerpt of the resulting temporary collection.
Similarly to previous operators, the `DROP OTHERS` specification means that objects come out from the spatial join and not selected by the `CASE WHERE` condition must be dropped.
- Finally, the temporary collection produced by the `SPATIAL JOIN` operator is saved with name `GeoCodedMessages` into database `Portal`, which is the storage area of the ElasticSearch service on server 10.0.0.13.

As the reader can see, J-CO-QL provides a unified view of NoSQL database systems which to work with. Second, the operators are high-level operators, completely independent of specific query languages provided by specific NoSQL database systems. Furthermore, the support for spatial operations is a native feature of J-CO-QL operators.

Analysts and data scientists can focus their minds on the problem to solve, thinking about the flow of transformations; they are unaware about low-level technical details concerning interfaces, APIs, and so on. Notice that J-CO-QL is not the first language proposed for querying and manipulating collections of JSON objects. Among many proposals, we mention *Jaql* (Beyer et al. (2009)) and *SQL++* (Ong et al. (2014)): they are good languages, but w.r.t. J-CO-QL they completely lack support for spatial operations and do not deal with heterogeneous objects at the same time (an important feature of J-CO-QL, well discussed in Bordogna et al. (2018)).

Also notice that support for spatial querying is necessary, now, in many contexts. For example, in the field of semantic web, the *GeoSPARQL* language (Battle et al. (2011)) has extended *SPARQL* (the RDF query language) with spatial querying capabilities.

Input Collection:
CountryBoundaries@AuthoritativeData

```
[{ "type": "FeatureCollection",
  "name": "EU_Countries",
  "features": [
    { "type": "Feature",
      "properties": { "Name": "Italy" },
      "geometry": { "type": "MultiPolygon",
                    "coordinates": [...] } },
    ... ]
  },
  ... ]
```

After EXPAND

```
[{ "type": "FeatureCollection",
  "name": "EU_Countries",
  "feature":
    { "item":
      { "type": "Feature",
        "properties": { "Name": "Italy" },
        "geometry": { "type": "MultiPolygon",
                      "coordinates": [...] } }
    },
    "position": 1 }
  },
  ... ]
```

After FIND

```
[{ "CountryName": "Italy",
  "~geometry": { "type": "MultiPolygon",
                  "coordinates": [...] } },
  ... ]
```

Input Collection:
Messages@SocialMedia

```
[{ "postID": "1000010001000",
  "userID": "Falcon9",
  "date": "2018-0801",
  "text": "Wow, wonderful Venice",
  "~geometry": { "type": "point",
                  "coordinates": [12.337575,
                                  45.434176 ] }
  },
  ... ]
```

Within SPATIAL JOIN

```
[{ "M": { "postID": "1000010001000",
  "userID": "Falcon9",
  "date": "2018-0801",
  "text": "Wow, wonderful Venice",
  "~geometry": { "type": "point",
                  "coordinates": [12.337575,
                                  45.434176 ] }
  },
  "C": { "CountryName": "Italy",
  "~geometry": { "type": "MultiPolygon",
                  "coordinates": [...] } },
  "~geometry": { "type": "point",
                  "coordinates": [12.337575,
                                  45.434176 ] }
  }, ... ]
```

Aftern SPATIAL JOIN

```
[{ "postID": "1000010001000",
  "userID": "Falcon9",
  "date": "2018-0801",
  "text": "Wow, wonderful Venice",
  "countryName": "Italy",
  "coordinates": [12.337575, 45.434176 ]
  }, ... ]
```

Figure 2. Input collections and intermediate results of the query

4. CONCLUSION

In this paper, we motivate the need for a polystore framework that allows analysts to easily manage geo-data sets represented as collections of JSON documents and as GeoJSON information layers. The wide availability of such a kind of data sets, joined with the availability of NoSQL database systems that directly work with JSON data sets, such as MongoDB and ElasticSearch, demands for an approach based on the polystore paradigm. The goal is to provide analysts with a high-level tool able to perform complex transformations on JSON geo-data sets, in a way independent of the specific NoSQL database system used to store them.

The proposed framework, named J-CO, is based on this approach. The J-CO-QL query language is independent of the storage platform and provides high-level operators for performing complex transformations, including spatial transformations. The J-CO-QL Engine is able to connect with various NoSQL database system in a seamless way. The J-CO-DS is able to create a database view of large JSON data sets stored in a file system, in order to overcome limitations of other NoSQL database systems and increase the flexibility of the framework.

Both the polystore paradigm and the J-CO-DS service are the novelty of this paper w.r.t. our previous research on the framework (Bordogna et al. (2017b), Bordogna et al. (2018)).

Until now, we spoke about various advantages provided by the J-CO framework. Obviously, the level of flexibility provided by the framework introduces the main issue of execution performance. In fact, the J-CO-QL Engine has to transfer possibly huge data sets through the Intranet from database systems, process them and transfer results to other database systems. Although the transfer time cannot be dropped (but this is done by the engine in place of users), the processing time can be optimized. We are starting investigate how to optimize the execution of spatial joins, by building temporary spatial indexes.

The next steps in our research will be primarily the following ones. First of all, we will complete the current version of the framework with a user interface that effectively aids analysts to write complex queries. The idea is that an analyst interactively writes a query, step-by-step, by inspecting the content of the temporary collection; if the results are not satisfying, the query process can be rolled back (or back-tracked) to a previous state and modified. In order to deal with very-large data sets, we are going to develop a new version of components in the J-CO-QL Engine based on the Map-Reduce paradigm and Hadoop (White (2011)).

However, the major evolution will be the definition of a kind of meta query language. In fact, J-CO-QL is not exactly easy, because it is necessary to precisely know the actual structure of JSON objects to transform. Our long-term goal is to push even higher the level of querying, so that analysts can formulate queries without knowledge of the actual structure of objects. This is why we speak about a meta query language; in order to define it, we will rely our experience on uncertain location-based spatial queries (Bordogna et al. (2008c), Bordogna et al. (2009)), on languages for manipulating and clustering web search results (Bordogna et al. (2008), Bordogna et al. (2008b)) and large sets of tweets of moving users (Bordogna et al. (2016)).

REFERENCES

- Banker, K. (2011). MongoDB in action. Manning Publications Co..
- Battle, R., and Kolas, D. (2011). Geosparql: enabling a geospatial semantic web. *Semantic Web Journal*, 3(4), 355-370.
- Beyer, K., Ercegovac, V., Rao, J., and Shekita, E. (2009). Jaql: A JSON query language. URL: <http://jaql.org>.
- Bordogna, G., Pagani, M., and Psaila, G. (2006). Database model and algebra for complex and heterogeneous spatial entities. In *Progress in Spatial Data Handling* (pp. 79-97). Springer, Berlin, Heidelberg.
- Bordogna, G., Campi, A., Psaila, G., and Ronchi, S. (2008). A language for manipulating clustered web documents results. In *Proceedings of the 17th ACM conference on Information and knowledge management* (pp. 23-32). ACM.
- Bordogna, G., Campi, A., Psaila, G., and Ronchi, S. (2008b). An interaction framework for mobile web search. In *Proceedings of the 6th International Conference on Advances in Mobile Computing and Multimedia* (pp. 183-191). ACM.
- Bordogna, G., Pagani, M., Pasi, G., and Psaila, G. (2008c). Evaluating uncertain location-based spatial queries. In *Proceedings of the 2008 ACM symposium on Applied computing* (pp. 1095-1100). ACM.

- Bordogna, G., Pagani, M., Pasi, G., and Psaila, G. (2009). Managing uncertainty in location-based queries. *Fuzzy sets and systems*, 160(15), 2241-2252.
- Bordogna, G., Frigerio, L., Cuzzocrea, A., and Psaila, G. (2016). Clustering geo-tagged tweets for advanced big data analytics. In *Big Data (BigData Congress), 2016 IEEE International Congress on* (pp. 42-51). IEEE.
- Bordogna, G., Capelli, S., and Psaila, G. (2017). A big geo data query framework to correlate open data with social network geotagged posts. In *The Annual International Conference on Geographic Information Science* (pp. 185-203). Springer, Cham.
- Bordogna, G., Ciriello, D. E., and Psaila, G. (2017b). A flexible framework to cross-analyze heterogeneous multi-source geo-referenced information: the J-CO-QL proposal and its implementation. In *Proceedings of the International Conference on Web Intelligence* (pp. 499-508). ACM.
- Bordogna, G., Capelli, S., Ciriello, D. E., and Psaila, G. (2018). A cross-analysis framework for multi-source volunteered, crowdsourced, and authoritative geographic information: the case study of volunteered personal traces analysis against transport network data. *Geo-spatial Information Science*, 21 (3), 257-271.
- Bryan, P., and Nottingham, M. (2013). *JavaScript Object Notation (JSON) Patch* (No. RFC 6902).
- Burini, F., Ciriello, D. E., Ghisalberti, A., and Psaila, G. (2018). The urban nexus project: when urban mobility analysis, vgi and data science meet together. In *Mobile Information Systems Leveraging Volunteered Geographic Information for Earth Observation* (pp. 111-130). Springer, Cham.
- Burini, F., Cortesi, N., Gotti, K., and Psaila, G. (2018b). The Urban Nexus Approach for Analyzing Mobility in the Smart City: Towards the Identification of City Users Networking. *Mobile Information Systems, Volume 2018, Article ID 6294872*,.
- Butler, H., Daly, M., Doyle, A., Gillies, S., Hagen, S., and Schaub, T. (2016). *The GeoJSON format* (No. RFC 7946).
- Cattell, R. (2011). Scalable SQL and NoSQL data stores. *Acm Sigmod Record*, 39(4), 12-27.
- Chowdhury, G. G. (2010). Introduction to modern information retrieval. Facet publishing.
- Cuzzocrea, A., Psaila, G., and Toccu, M. (2015). Knowledge discovery from geo-located tweets for supporting advanced big data analytics: A real-life experience. In *Model and Data Engineering* (pp. 285-294). Springer, Cham.
- Cuzzocrea, A., Psaila, G., and Toccu, M. (2016). An innovative framework for effectively and efficiently supporting big data analytics over geo-located mobile social media. In *Proceedings of the 20th International Database Engineering & Applications Symposium* (pp. 62-69). ACM.
- Duggan, J., Elmore, A. J., Stonebraker, M., Balazinska, M., Howe, B., Kepner, J., and Zdonik, S. (2015). The bigdawg polystore system. *ACM Sigmod Record*, 44(2), 11-16.
- ESRI, E. Shapefile technical description, jul. 1998.
- Gessert, F., Wingerath, W., Friedrich, S., and Ritter, N. (2017). NoSQL database systems: a survey and decision guidance. *Computer Science-Research and Development*, 32(3-4), 353-365.
- Gormley, C., and Tong, Z. (2015). Elasticsearch: The Definitive Guide: A Distributed Real-Time Search and Analytics Engine. " O'Reilly Media, Inc.". Han, J., Haihong, E., Le, G., & Du, J. (2011). Survey on NoSQL database. In *Pervasive computing and applications (ICPCA), 2011 6th international conference on* (pp. 363-366). IEEE.
- Latif, A., Saeed, A. U., Hoefler, P., Stocker, A., and Wagner, C. (2009, September). The Linked Data Value Chain: A Lightweight Model for Business Engineers. In *I-SEMANTICS* (pp. 568-575).
- Nayak, A., Poriya, A., and Poojary, D. (2013). Type of NOSQL databases and its comparison with relational databases. *International Journal of Applied Information Systems*, 5(4), 16-19.
- Ong, K. W., Papakonstantinou, Y., and Vernoux, R. (2014). The SQL++ unifying semi-structured query language, and an expressiveness benchmark of SQL-on-Hadoop, NoSQL and NewSQL databases. *CoRR*, abs/1405.3631.
- Psaila, G. (2011). A database model for heterogeneous spatial collections: Definition and algebra. In *Data and Knowledge Engineering (ICDKE), 2011 International Conference on* (pp. 30-35). IEEE.
- Smith, B. (2015). Introducing JSON. In *Beginning Json* (pp. 37-47). Apress, Berkeley, CA.
- White, T. (2012). *Hadoop: The definitive guide*. " O'Reilly Media, Inc.".